

LARGE LANGUAGE MODELS

LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models

M2 IASD

Monday 8th June, 2026



LLaMA-Factory

Easy and Efficient LLM Fine-Tuning

Sepanta Farzollahi
Melissa Hafis

1 INTRODUCTION

2 THE PAPER: LLAMAFACTORY

3 OUR REPLICATION

4 NOVEL ABLATION

5 CONCLUSION

6 REFERENCES

INTRODUCTION

Why Fine-Tuning Matters

Pre-trained LLMs are powerful but must be adapted to specific tasks. Full fine-tuning of a 7B-parameter model requires ~ 38 GB of GPU memory and days of compute — prohibitive for most practitioners.

Existing Solutions

- **LoRA** [1]: low-rank adapter matrices
- **QLoRA** [2]: LoRA on 4-bit base
- **GaLore** [3]: gradient low-rank projection
- **Freeze-tuning** [4]: partial layer update

Research Gap

Each method is implemented independently with different interfaces, making fair, reproducible comparison across model families very difficult.

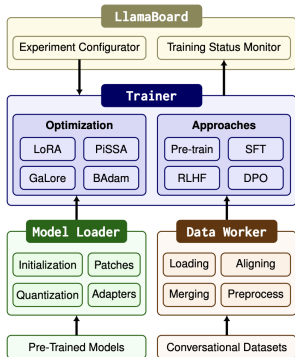
Answer: LlamaFactory [5]

A unified framework integrating 100+ LLMs and all major efficient fine-tuning methods under a single CLI and optional web interface (Demo).

THE PAPER: LLAMAFACTORY

Core Idea

One framework, one API, for all methods and all models. Users configure experiments via YAML files or a web UI — no custom code per model.



Supported Training Objectives

- Supervised Fine-Tuning (SFT)
- Reinforcement Learning from Human Feedback (RLHF)
- Direct Preference Optimization (DPO)
- Generative Pre-training

Impact

>25,000 GitHub stars | >3,000 forks | Hundreds of models on Hugging Face Hub built with LlamaFactory.

Optimization Methods

Which parameters to update?

- **Full fine-tuning** — all parameters updated, maximum expressivity
- **Freeze-tuning** — last k decoder layers only [4]
- **GaLore** — all params, gradients projected to low-rank subspace [3]
- **LoRA** — frozen base + trainable low-rank A , B matrices [1]
- **QLoRA** — LoRA on NF4 4-bit quantized base model [2]

Computation Methods

How to make each update cheaper?

- Mixed precision (fp16 / bf16)
- Activation checkpointing
- Flash Attention
- 4-bit NF4 quantization (bitsandbytes)
- Unsloth: Triton-accelerated LoRA backward pass

Key Trade-off

Optimization methods reduce the *number* of parameters updated. Computation methods reduce the *cost* of each update. LlamaFactory combines both transparently.

Original Setup (Zheng et al., 2024)

Gemma-2B, Llama2-7B, Llama2-13B trained on PubMed (~400K tokens), generative pre-training objective, single NVIDIA A100 40 GB.

Method	Gemma-2B				Llama2-7B				Llama2-13B			
	Params	Mem	Tput	PPL	Params	Mem	Tput	PPL	Params	Mem	Tput	PPL
Baseline	/	/	/	11.83	/	/	/	7.53	/	/	/	6.66
Full-FT	2.51B	17.1	3090	10.34	6.74B	38.7	1335	5.56	/	/	/	/
Freeze	0.33B	8.10	5608	11.33	0.61B	15.7	2905	6.59	0.95B	29.0	1841	6.56
GaLore	2.51B	10.2	2483	10.38	6.74B	15.4	1584	5.88	13.0B	28.9	956	5.72
LoRA	0.16B	7.91	3521	10.19	0.32B	16.3	1954	5.81	0.50B	30.1	1468	5.75
QLoRA	0.16B	5.21	3159	10.46	0.32B	7.52	1579	5.91	0.50B	12.6	974	5.81

Three Key Patterns

- **QLoRA** → lowest memory
- **LoRA** → best throughput and PPL (Perplexity)
- **Full-FT** → best PPL on small models

OUR REPLICATION

Replication Goal

Reproduce the **qualitative patterns** of Table 4 on a single consumer GPU and add a **novel ablation** not present in the original paper.

Setup

- **Model:** TinyLlama-1.1B-Chat-v1.0
- **Dataset:** Alpaca GPT-4 (SFT, 10,000 examples)
- **GPU:** NVIDIA RTX 6000 Ada (VRAM: 50.9 GB)
- **Epochs:** 3
- **Seq. length:** 512 tokens
- **Precision:** fp16

Six Conditions

- Baseline (zero-shot PPL, no training)
- Full fine-tuning
- Freeze-tuning (last 4 layers)
- GaLore (rank 16, scale 0.25)
- LoRA (rank 128, $\alpha \approx 256$, all layers)
- QLoRA (rank 128, 4-bit NF4)

Critical Implementation: Correct Baseline Evaluation

The baseline PPL uses the **Zephyr chat template** with **response-only loss** and **padding masking** — matching LlamaFactory's SFT evaluation protocol exactly. A naive implementation gives PPL $\approx 90,000$.

10,000 Examples, TinyLlama-1.1B (best validation checkpoint)

All qualitative patterns from Table 4 of the paper are successfully reproduced.

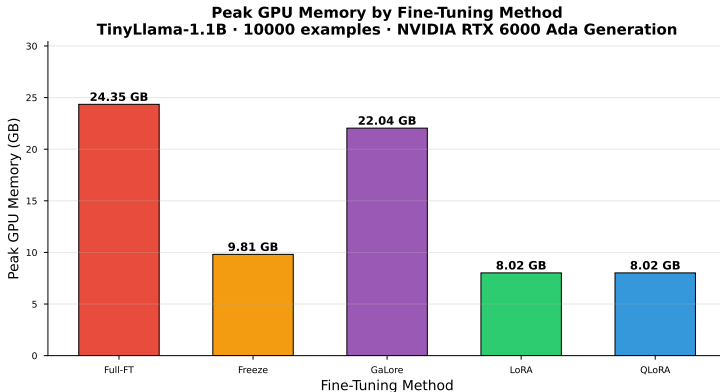
Method	Trainable Params	Memory (GB)	Throughput (tok/s)	Best Eval Loss	PPL
Baseline	/	/	/	/	3.51
Full-FT	1,100M	24.35	6,981	1.109	3.03
Freeze	176M	9.81	16,836	1.134	3.11
GaLore	1,100M	22.04	6,838	1.123	3.07
LoRA	18M	8.02	10,440	1.117	3.06
QLoRA	18M	8.02	10,454	1.117	3.06

Confirmed Patterns

- All methods improve over Baseline (3.51 → 3.03–3.11)
- LoRA/QLoRA: best PPL with only 18M trainable parameters (<2%)
- Freeze: highest throughput (16,836 tok/s), modest PPL gain

Difference vs. Paper

At 1.1B scale, LoRA and QLoRA achieve **identical PPL**. Quantization noise is negligible here — the gap documented in the paper (7–13B models) does not appear at smaller scale.



Memory Hierarchy

Full-FT (24.4 GB) > GaLore (22.0 GB) > Freeze (9.8 GB) > LoRA $\approx$ QLoRA (8.0 GB)

GaLore Note

Saves optimizer-state memory vs. Full-FT but still stores all fp16 weights. The gap vs. Full-FT widens on larger models.



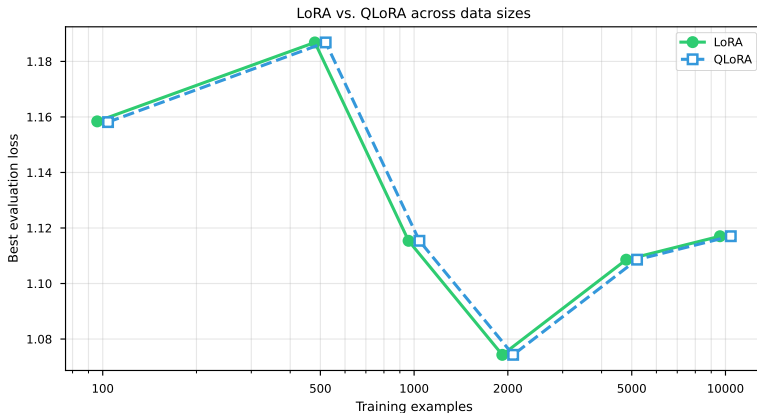
Observations

All methods converge smoothly over 3 epochs. Full-FT and GaLore show a slightly faster early descent (all parameters updated), while LoRA/QLoRA reach competitive final loss with $60\times$ fewer trainable parameters. Freeze-tuning converges fastest but reaches a higher final loss floor.

NOVEL ABLATION

Research Question

Does the performance gap between LoRA and QLoRA widen when training data is scarce? We test 6 dataset scales: 100, 500, 1,000, 2,000, 5,000, and 10,000 examples.



Quantitative Results

Examples	LoRA Loss	LoRA PPL	QLoRA Loss	QLoRA PPL	Gap (Q-L)
100	1.1584	3.18	1.1581	3.18	-0.0003
500	1.1868	3.28	1.1868	3.28	≈ 0
1,000	1.1154	3.05	1.1154	3.05	≈ 0
2,000	1.0743	2.93	1.0743	2.93	≈ 0
5,000	1.1086	3.03	1.1086	3.03	≈ 0
10,000	1.1170	3.06	1.1170	3.06	≈ 0

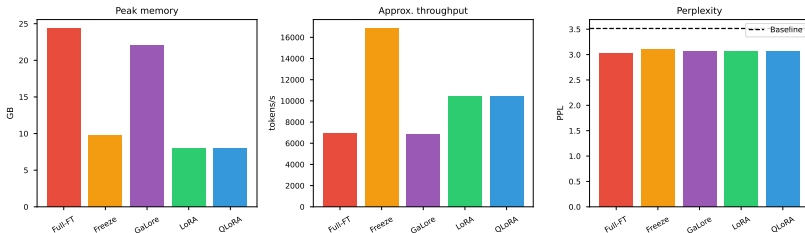
Finding

At 1.1B scale, LoRA and QLoRA are **statistically indistinguishable** at every data size. The maximum absolute gap is 0.0003 in eval loss.

Interpretation

The quantization-noise hypothesis from the paper holds **only at large scale** ($\geq 7B$). At 1.1B, 4-bit NF4 noise is too small to affect task performance regardless of training data volume.

Main Experiment Summary · TinyLlama-1.1B · 10000 examples · NVIDIA RTX 6000 Ada Generation



Best Quality-Efficiency Trade-off: LoRA and QLoRA

- Competitive PPL (3.06) vs. Full-FT (3.03) with 60× fewer trainable parameters
- 2.7× less memory than GaLore for the same parameter count
- 1.5× higher throughput than Full-FT

CONCLUSION

What We Confirmed

All qualitative patterns from Table 4 of Zheng et al. (2024) are reproduced on TinyLlama-1.1B with a single GPU: QLoRA has the lowest memory footprint; LoRA and QLoRA achieve the best quality–efficiency trade-off; Full-FT maximises quality at high memory cost; Freeze-tuning is fastest but least expressive.

Novel Finding

At 1.1B scale, LoRA and QLoRA are **equivalent in quality** across all dataset sizes (100–10,000 examples). The quantization penalty documented in the paper emerges only at $\geq 7\text{B}$ parameter scale.

Limitations

- Single seed — no confidence intervals
- TinyLlama-1.1B $\neq$ 7–13B (trends may not transfer)
- Alpaca SFT $\neq$ PubMed pre-training (different task)
- Throughput estimated, not profiled with a dedicated tool

LlamaFactory

A mature unified framework that makes fair comparison of efficient fine-tuning methods trivial — over 25,000 stars, 100+ supported models, one YAML per experiment.

Our Replication Study

- Reproduced all Table 4 patterns on TinyLlama-1.1B (NVIDIA RTX 6000 Ada)
- Implemented 6 conditions including a carefully calibrated baseline (Zephyr template, response-only loss, padding masking)
- Added a novel ablation across 6 dataset sizes (100–10,000 examples)

Take-Away Message

LoRA is the best general-purpose choice at sub-2B scale: it matches Full-FT quality with $60\times$ fewer trainable parameters, $2.7\times$ less memory than GaLore, and $1.5\times$ higher throughput. QLoRA adds no quality penalty at this scale — use it when VRAM is limited.

REFERENCES

- [1] Edward J Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations*. 2022.
- [2] Tim Dettmers et al. “QLoRA: Efficient Fine-Tuning of Quantized LLMs”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 10088–10115.
- [3] Jiawei Zhao et al. “GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection”. In: *International Conference on Machine Learning*. 2024.
- [4] Neil Houlsby et al. “Parameter-Efficient Transfer Learning for NLP”. In: *International Conference on Machine Learning*. 2019, pp. 2790–2799.
- [5] Yaowei Zheng et al. “LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, 2024, pp. 400–410.